Python Activity

Simplified Python case studies (25–34)

CASE STUDY 25: Progressive Water Tariff Analysis

1. Problem Statement

Calculate water bills for households based on water usage. Add a fixed fee and a penalty for high usage.

2. Objectives

Calculate the bill, apply the extra penalty when needed, and find the household with the highest usage.

3. Algorithm

1. Read the number of households.

2. For each household, read the name and water units.

3. Calculate the charge using the slabs: first 10 units at Rs. 5 each, next 10 at Rs. 8 each, and remaining units at Rs. 12 each.

4. Add Rs. 30 fixed fee. If usage is above 30 units, add Rs. 50 penalty.

5. Display each bill and the household with the highest usage.

4. Simple Python Code

```
n = int(input("Number of households: "))
highest_name = ""
highest_units = -1

for i in range(n):
    name = input("Household name: ")
    units = int(input("Water units used: "))

    if units < 0:
        print("Invalid units")
        continue

    if units <= 10:
        charge = units * 5
    elif units <= 20:
        charge = 10 * 5 + (units - 10) * 8
    else:
        charge = 10 * 5 + 10 * 8 + (units - 20) * 12

    bill = charge + 30
    if units > 30:
        bill = bill + 50
        print("Excessive usage!")

    print(name, "bill = Rs.", bill)

    if units > highest_units:
        highest_units = units
        highest_name = name

if highest_units >= 0:
    print("Highest usage:", highest_name, highest_units, "units")
```

5. Code Explanation

- The if-elif-else statements calculate the charge for the correct usage slab.

- Rs. 30 is added to every valid bill. Usage above 30 units gets an extra Rs. 50.

- The highest_units variable keeps track of the largest usage entered.

6. Sample Output

```
Number of households: 2
Household name: Asha
Water units used: 8
Asha bill = Rs. 70
Household name: Chitra
Water units used: 35
Excessive usage!
Chitra bill = Rs. 410
Highest usage: Chitra 35 units
```

7. Conclusion

The program calculates progressive water bills, adds fees and penalties, and identifies the highest usage.

CASE STUDY 26: Examination Seating Generator

1. Problem Statement

Create a seating plan using row and column numbers. Reserved seats are marked R, alternate empty seats are marked X, and available seats are marked S.

2. Objectives

Use nested loops, mark reserved seats, and print the seating plan.

3. Algorithm

1. Read the number of rows and columns.

2. Check that both values are positive.

3. Use one loop for rows and another loop for columns.

4. Print R for reserved seats, X for alternate empty seats, and S for other seats.

4. Simple Python Code

```
rows = int(input("Number of rows: "))
cols = int(input("Number of columns: "))

reserved = [(1, 2), (2, 3)]

if rows <= 0 or cols <= 0:
    print("Invalid classroom size")
else:
    for row in range(1, rows + 1):
        for col in range(1, cols + 1):
            if (row, col) in reserved:
                print("R", end=" ")
            elif (row + col) % 2 == 0:
                print("X", end=" ")
            else:
                print("S", end=" ")
        print()

    print("S = Available, R = Reserved, X = Empty")
```

5. Code Explanation

- The outer loop moves through each row.

- The inner loop checks every column in that row.

- The reserved-seat check comes first, so reserved seats are always marked R.

6. Sample Output

```
Number of rows: 2
Number of columns: 3
X R X
S X R
S = Available, R = Reserved, X = Empty
```

7. Conclusion

Nested loops make it easy to create a seating chart and apply seat rules.

CASE STUDY 27: Hollow Diamond Pattern

1. Problem Statement

Print a hollow diamond using stars. The size must be between 1 and 20.

2. Objectives

Use loops and spaces to create a hollow diamond and reject invalid sizes.

3. Algorithm

1. Read the size.

2. If it is outside 1 to 20, display an error.

3. Print the upper half and then the lower half of the diamond.

4. Simple Python Code

```
n = int(input("Enter diamond size (1-20): "))

if n < 1 or n > 20:
    print("Invalid size")
else:
    for i in range(n):
        spaces = n - i - 1
        if i == 0:
            print(" " * spaces + "*")
        else:
            inside = 2 * i - 1
            print(" " * spaces + "*" + " " * inside + "*")

    for i in range(n - 2, -1, -1):
        spaces = n - i - 1
        if i == 0:
            print(" " * spaces + "*")
        else:
            inside = 2 * i - 1
            print(" " * spaces + "*" + " " * inside + "*")
```

5. Code Explanation

- The first loop prints the upper half of the diamond.

- The second loop prints the lower half in reverse order.

- The spaces inside each row keep the diamond hollow.

6. Sample Output

```
Enter diamond size (1-20): 3
  *
 * *
*   *
 * *
  *
```

7. Conclusion

The program uses loops and spaces to print a hollow diamond pattern.

CASE STUDY 28: Repeated Transaction Detection

1. Problem Statement

Read transaction IDs and amounts. Report duplicate IDs and amounts greater than Rs. 10,000.

2. Objectives

Store IDs already seen, detect repeated IDs, and flag large amounts.

3. Algorithm

1. Read the number of transactions.

2. For each transaction, read its ID and amount.

3. Check whether the ID has appeared before and whether the amount is above Rs. 10,000.

4. Display whether the transaction is normal or suspicious.

4. Simple Python Code

```
n = int(input("Number of transactions: "))
seen = set()

for i in range(n):
    tid = input("Transaction ID: ")
    amount = float(input("Amount: "))
    reasons = []

    if tid in seen:
        reasons.append("duplicate ID")
    seen.add(tid)

    if amount > 10000:
        reasons.append("amount above Rs. 10000")

    if reasons:
        print("Suspicious:", tid, amount, "-", ", ".join(reasons))
    else:
        print("Normal:", tid, amount)
```

5. Code Explanation

- The set named seen stores transaction IDs that have already been entered.

- The reasons list stores any warning for the current transaction.

- If there are no warnings, the transaction is displayed as normal.

6. Sample Output

```
Number of transactions: 3
Transaction ID: T01
Amount: 500
Normal: T01 500.0
Transaction ID: T01
Amount: 700
Suspicious: T01 700.0 - duplicate ID
Transaction ID: T03
Amount: 15000
Suspicious: T03 15000.0 - amount above Rs. 10000
```

7. Conclusion

The program finds duplicate transaction IDs and unusually large amounts.

CASE STUDY 29: Password Validation

1. Problem Statement

Check a password for length, uppercase and lowercase letters, a digit, a special character, repeated characters, and the prohibited word 'password'.

2. Objectives

Check password rules and explain which rules are not satisfied.

3. Algorithm

1. Read the password.

2. Check each rule and save any errors in a list.

3. If the list is empty, display that the password is valid. Otherwise, display the errors.

4. Simple Python Code

```
import string

password = input("Enter password: ")
errors = []

if len(password) < 8:
    errors.append("must have at least 8 characters")
if not any(ch.isupper() for ch in password):
    errors.append("needs an uppercase letter")
if not any(ch.islower() for ch in password):
    errors.append("needs a lowercase letter")
if not any(ch.isdigit() for ch in password):
    errors.append("needs a digit")
if not any(ch in string.punctuation for ch in password):
    errors.append("needs a special character")

for i in range(len(password) - 2):
    if password[i] == password[i + 1] == password[i + 2]:
        errors.append("three repeated characters are not allowed")
        break

if "password" in password.lower():
    errors.append("must not contain the word 'password'")

if errors:
    print("Invalid password:")
    for error in errors:
        print("-", error)
else:
    print("Valid password")
```

5. Code Explanation

- len(password) checks the number of characters.

- The any() checks look for uppercase letters, lowercase letters, digits, and special characters.

- The final checks reject three same characters in a row and the word password, ignoring letter case.

6. Sample Output

```
Enter password: abc
Invalid password:
- must have at least 8 characters
- needs an uppercase letter
- needs a digit
- needs a special character
```

7. Conclusion

The program checks several password rules and shows the reasons for an invalid password.

CASE STUDY 30: Student Email and ID Validation

1. Problem Statement

Check whether a student email and registration ID follow simple formatting rules.

2. Objectives

Check email structure, check registration ID structure, and show valid or invalid results.

3. Algorithm

1. Read the email and registration ID.

2. Check that the email has one @, a dot after @, and no spaces.

3. Check that the ID has two uppercase letters followed by four digits.

4. Display the results.

4. Simple Python Code

```
email = input("Enter student email: ")
reg_id = input("Enter registration ID: ")

if (email.count("@") == 1 and " " not in email
        and "." in email.split("@")[1]):
    print("Email: Valid")
else:
    print("Email: Invalid format")

if (len(reg_id) == 6 and reg_id[:2].isalpha()
        and reg_id[:2].isupper() and reg_id[2:].isdigit()):
    print("Registration ID: Valid")
else:
    print("Registration ID: Invalid format")
```

5. Code Explanation

- count('@') checks that the email contains exactly one @ symbol.

- split('@')[1] gets the part after @, where a dot must appear.

- The ID check requires six characters: two uppercase letters and four digits.

6. Sample Output

```
Enter student email: student@college.in
Enter registration ID: CS2026
Email: Valid
Registration ID: Valid
```

7. Conclusion

The program uses simple string checks to validate email and registration ID formats.

CASE STUDY 31: Examination Answer Evaluation

1. Problem Statement

Compare student answers with an answer key. Count correct, incorrect, and blank answers and calculate the score with negative marking.

2. Objectives

Compare answers, count each type of response, and calculate the final score.

3. Algorithm

1. Read the answer key and student answers separated by spaces. Use - for a blank answer.

2. Check that both lists have the same length.

3. Compare each answer with the key.

4. Give 1 mark for a correct answer, subtract 0.25 for a wrong answer, and give 0 for a blank.

4. Simple Python Code

```
key = input("Answer key (use spaces): ").upper().split()
answers = input("Student answers (use - for blank): ").upper().split()

if len(key) != len(answers):
    print("Error: answer counts do not match")
else:
    correct = 0
    wrong = 0
    blank = 0

    for k, answer in zip(key, answers):
        if answer == "-":
            blank += 1
        elif answer == k:
            correct += 1
        else:
            wrong += 1

    score = correct - wrong * 0.25
    print("Correct:", correct)
    print("Incorrect:", wrong)
    print("Unanswered:", blank)
    print("Final score:", score)
```

5. Code Explanation

- split() changes the entered answers into lists.

- zip() compares each key answer with the matching student answer.

- The score is correct answers minus 0.25 for every wrong answer.

6. Sample Output

```
Answer key (use spaces): A B C D
Student answers (use - for blank): A B - A
Correct: 2
Incorrect: 1
Unanswered: 1
Final score: 1.75
```

7. Conclusion

The program counts answers and calculates the score using negative marking.

CASE STUDY 32: Invoice Data Extraction

1. Problem Statement

Read an invoice record in CODE,QUANTITY,AMOUNT format and display its details. Reject records with incorrect data.

2. Objectives

Split the record into fields, check the values, and report invalid records.

3. Algorithm

1. Read the invoice record.

2. Split it using commas and check that there are three fields.

3. Convert quantity to an integer and amount to a number.

4. Check that quantity is positive and amount is not negative.

4. Simple Python Code

```
record = input("Enter invoice record (CODE,QUANTITY,AMOUNT): ")
fields = record.split(",")

if len(fields) != 3:
    print("Malformed invoice record")
else:
    code = fields[0].strip()

    try:
        quantity = int(fields[1].strip())
        amount = float(fields[2].strip())

        if code == "" or quantity <= 0 or amount < 0:
            print("Malformed invoice record")
        else:
            print("Product code:", code)
            print("Quantity:", quantity)
            print("Amount: Rs.", amount)
    except ValueError:
        print("Malformed invoice record: quantity or amount is not numeric")
```

5. Code Explanation

- split(',') separates the code, quantity, and amount.

- strip() removes extra spaces from each field.

- try and except handle values that cannot be converted into numbers.

6. Sample Output

```
Enter invoice record (CODE,QUANTITY,AMOUNT): P101,2,150.50
Product code: P101
Quantity: 2
Amount: Rs. 150.5
```

7. Conclusion

The program extracts invoice details and handles incorrectly formatted records.

CASE STUDY 33: Campus Registration ID Analyzer

1. Problem Statement

Check a campus registration ID and separate its branch, admission year, and student number.

2. Objectives

Validate the ID format, extract its parts, and display the result.

3. Algorithm

1. Read the registration ID.

2. Check that it has 10 characters.

3. Check for three uppercase letters, four digits for the year, and three digits for the student number.

4. Display the parts if valid; otherwise display an error.

4. Simple Python Code

```
reg_id = input("Enter registration ID: ")

if (len(reg_id) == 10 and reg_id[:3].isalpha()
        and reg_id[:3].isupper() and reg_id[3:7].isdigit()
        and reg_id[7:10].isdigit()):
    branch = reg_id[:3]
    year = reg_id[3:7]
    number = reg_id[7:10]

    print("Branch:", branch)
    print("Admission year:", year)
    print("Student number:", number)
else:
    print("Invalid registration ID")
```

5. Code Explanation

- len(reg_id) checks that the ID has 10 characters.

- Slicing [:3], [3:7], and [7:10] gets the branch, year, and number.

- isalpha(), isupper(), and isdigit() check the required character types.

6. Sample Output

```
Enter registration ID: CSE2026123
Branch: CSE
Admission year: 2026
Student number: 123
```

7. Conclusion

The program checks the registration ID format and separates its three parts.

CASE STUDY 34: Student Feedback Sentiment Scoring

1. Problem Statement

Read feedback and classify it as Positive, Negative, or Neutral by counting positive and negative keywords.

2. Objectives

Count keywords, calculate a score, and classify the feedback.

3. Algorithm

1. Read the feedback and convert it to lowercase.

2. Count positive and negative keywords.

3. Subtract the negative count from the positive count.

4. Classify a score above zero as Positive, below zero as Negative, and zero as Neutral.

4. Simple Python Code

```
feedback = input("Enter student feedback: ").lower()

positive_words = ["good", "helpful", "excellent", "clear"]
negative_words = ["bad", "confusing", "poor", "slow"]

positive_count = 0
negative_count = 0

for word in positive_words:
    if word in feedback:
        positive_count += 1

for word in negative_words:
    if word in feedback:
        negative_count += 1

score = positive_count - negative_count

if score > 0:
    sentiment = "Positive"
elif score < 0:
    sentiment = "Negative"
else:
    sentiment = "Neutral"

print("Positive keywords:", positive_count)
print("Negative keywords:", negative_count)
print("Score:", score)
print("Sentiment:", sentiment)
```

5. Code Explanation

- lower() makes the search ignore uppercase and lowercase differences.

- The loops count the positive and negative keywords found in the feedback.

- The score decides whether the feedback is Positive, Negative, or Neutral.

6. Sample Output

```
Enter student feedback: The class was good and clear
Positive keywords: 2
Negative keywords: 0
Score: 2
Sentiment: Positive
```

7. Conclusion

The program uses simple keyword counting to classify student feedback.